
FreeRTOS V6.1.1 Scheduler

通用形式化验证进展报告

Isabelle/HOL • generated C source • cursor-general semantics

严格闭环约 35–40%

粗粒度 gate 台账：暂定 $9/20 = 45\%$

两个口径必须分开；均不是 theory/session 数量比

报告日期：2026-08-11

本版基线：476e950；safe-loop leaf 已提交并推送

分支：agent/universal-scheduler-refinement

公开审阅：<https://github.com/hellotommmmy/freertos-v611-scheduler-isabelle/pull/2>

核验准则：Isabelle2025-2, quick_and_dirty=false, 无承认性证明

1 结论先行：已经完成多少？

审慎结论。若目标是旧的 fixed-witness / sealed milestone，则它已经完成；若目标是当前声明的“所有合法输入、任意合法 cursor、managed 与 runnable 分离、五根 generated-source 混合 trace、并发契约分离”的更强目标，则需要同时给出两种读数：**严格 acceptance closure 约 35–40%**；若只衡量已经可复用的表示、due-loop 与 tick 证明基础设施，则为**约 45–50%**。另按外部 Pro 建议的粗粒度 20-gate 台账可暂记为 $9/20 = 45\%$ ，但其中 A1–A8 尚须在仓库内逐项命名，且 `xTaskGetTickCount` 只能按 root-local 功能结果计分，不能冒充完整 common scheduler framing。上述数字都是审计口径，不是 Isabelle 定理，也不是把 371 个 theory 或 311 个 session 逐个计数所得。

1.1 为何不能按 theory/session 数量计算

- 一个 session 可能只证明一个 selector frame，也可能关闭一个 arbitrary-list generated while；二者语义权重不相等。 **A · audited**
- 同一语义缺口常被拆成多个小 session 以避免 Isabelle 搜索爆炸；拆分会增加文件数，却不会自动提高覆盖率。 **R · repository**
- “某个分支 checker-green”不等于“该根函数所有合法输入 checker-green”。例如 suspended-only switch、zero-delay 和固定 P2 delay 都是有效结果，但不应计成通用闭环。 **C · checked**
- 最终目标的负载门槛是：通用表示关系、每个根函数的 all-branch public closure、共用边界、混合有限 trace，以及与 generated source 相连的并发契约。 **A · audited**

1.2 两个同时成立的进度视角

视角	已获得的强证据	读数	解释
基础设施/单根深度	cursor-general 强快照、managed-domain due loop、完整 tick pipeline、公共边界、有限 tick trace	约 45–50%	大量最难的表示与 tick 控制流已闭合
粗粒度 20-gate 台账	暂按 A 6/8、B 2/5、C 0/4、D 1/3 汇总；A1–A8 名称仍待固化	$9/20 = 45\%$	可复算的暂定工程台账；不是 kernel 输出，也不是严格 closure 百分比
最终 acceptance gates	五根中只有 tick 与 tick-read 接近通用公共闭包；mixed trace 与 generated concurrency 仍开放	约 35–40%	只有 tick 具有完整 common scheduler framing；tick-read 仍需 framing adapter

三个读数回答不同问题，不应再合成一个看似精确的小数。“45–47%”只可作为带公开分母的 engineering-progress 台账或敏感性区间；对外回答“全部形式化证明完成多少”时，应优先使用

35–40% 的严格闭环区间，并同时列出门槛表。若第三方复算 45%，必须先把 A1–A8 逐项命名，并明确 tick-read 的 framing 缺口。 **A · audited**

2 证明架构与语义门槛

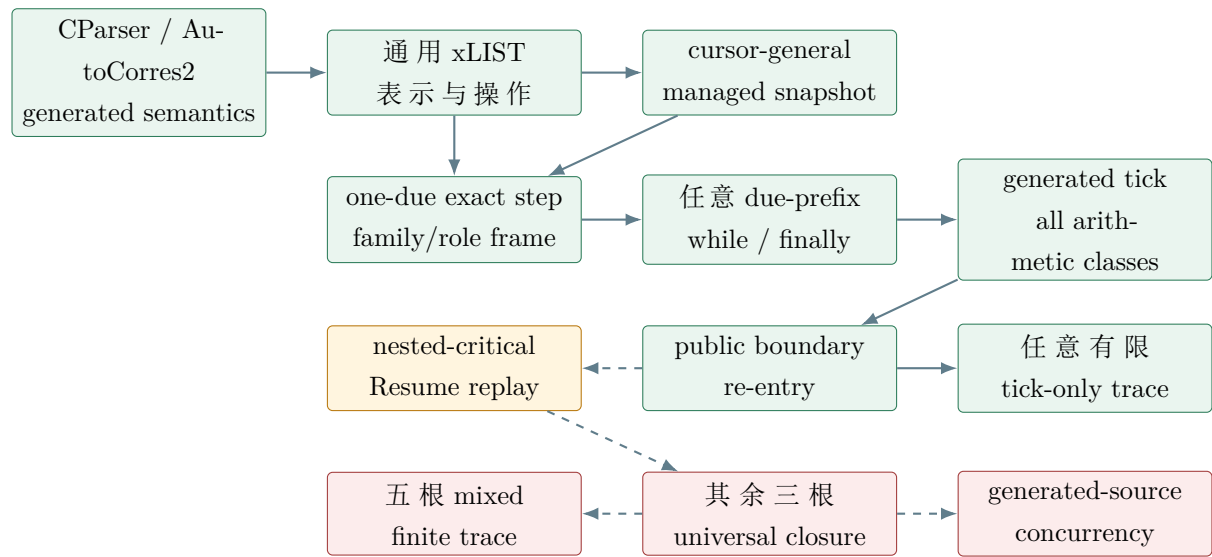


图 1: theorem/coverage DAG。绿色为 checker-green 类别，黄色为当前接缝，红色为未闭合 acceptance gate。

2.1 门槛表

门槛	定义	当前读数	解释
A	list primitives、byte geometry、family coverage、cursor-general 表示	6/8 (75%)	通用基础强，但不是完整 API closure
B	冻结五根函数的 all-branch public refinement	2/5 局部； 1/5 完整 framing	tick 完整；tick-read 的返回值正确但 relation 尚未携带 full scheduler state
C	共用 public invariant、五根 alphabet、mixed finite trace、跨根 closure	0/4	已有 tick-only seed，不等于五根 gate 完成
D	concurrent abstract interface、modular environment、generated connection	1/3	有基础设施；generated end-to-end 仍为 0
E	scope、工具链、QAD、排除项纪律	13/13 边界遵守	它是可信度条件，不计入证明覆盖率

A · audited 任何百分比都不应由上表直接做算术平均。A–D 的依赖、难度和 acceptance 含义不同；最诚实的呈现是“区间 + 离散门槛表 + 逐根状态”，而不是单一精确小数。

3 当前最关键的 checker-green 结果

以下每项均指结果类别; 只有仓库中实际存在且有绿色 session 证据的内容才标为 **C · checked**。早期 nested-tick probes 已晋升为正式的 checker-green staircase。

1. **通用 xLIST 与 raw/abstract 对齐**。任意合法 ring、节点、cursor、key 和 byte-heap 几何下, 删除、尾插、relabel、owner/container/key frame 具有精确关系。这些引理不固定任务身份或列表长度。 **C · checked**
2. **Generic/Event 全根 family coverage**。ready、delayed A/B、suspended、termination、pending 与任意有限 external Event roots 被统一量化; cross-family storage separation 在同一 heap cutpoint 重建。 **C · checked**
3. **managed M 与 runnable L 分离**。允许 termination 中存在 retired task, 维持 $L \subseteq M$, 避免旧 Gate-H 把 decoder carrier 错缩为 live set。 **C · checked**
4. **phase-aware due-loop invariant**。非空 due head 不再错误要求 stable core_wf/time_wf; due phase 允许 $key \leq now$, terminal 才恢复 stable snapshot。 **C · checked**
5. **一个 due task 的 exact generated step**。从真实 generated body 得到 exact Result pointer、精确最终 globals、family/role/observation/scalar/current/boundary frames, 并进入统一 successor witness。 **C · checked**
6. **任意有限 due-prefix while/finally**。覆盖零 due、singleton、任意 all-due、due+future; future head 的 body 为 exception, empty 为 ResultNULL, finally 统一归一为 Result()。 **C · checked**
7. **32-bit modular tick**。suspended missed counter 的 MAX_WORD- $\rightarrow 0$ 被保留; unlocked wrap 只在 literal signed guard 有定义时成功, signed-overflow 分支被证明无 successful run, 而不是被 no-wrap 假设排除。 **C · checked**
8. **cursor-general generated tick pipeline**。任意合法 delayed-list cursor、任意 managed/termination/external families 下, 从 generated prefix 经 arbitrary managed due loop 得到完整 cursor-general endpoint。 **C · checked**
9. **调用方隐藏 witness 的 public tick 边界闭包**。

CursorGeneralStrongScheduler
PublicBoundaryRel

该 relation 对调用方隐藏内部 witness 参数, 并证明成功 tick 重新建立同一 settled/unmasked public boundary。 **C · checked**

10. **nested-critical tick transport**。generated tick 对只改 proof-port depth/mask 的函数 overlay 无条件 self-bisimilar; 因此 Resume 内真实 1/1 cutpoint 可从 normalized 0/0 public shadow 运输, 而无需伪造 public boundary。 **C · checked**
11. **horizon-safe missed replay**。单次 body 严格按 tick 后 debt decrement; 递归 horizon 与 concrete signed-word guard 精确对应, 且 whole replay while 在 protected entry、quiet 与 full horizon 三项前提下完成并归零 debt。 **C · checked**

12. 任意有限 **repeated tick trace**. `cursor_general_vTaskIncrementTick_finite_trace` 对任意 n 、任意满足稳定性和逐步 arithmetic-enabled 的调用者关系, 给出 n 次 `task_increment_tick_modular_abs` 的精确迭代态。 **C · checked**

4 冻结五根函数的真实覆盖状态

根函数	最高已核验结果	仍缺什么	状态
<code>xTaskGetTickCount</code>	quiescent tick/depth/mask relation 下的返回值 refinement	full scheduler frame 与 common five-root boundary adapter	局部完整
<code>vTaskIncrementTick</code>	cursor-general public closure、all-arithmetic classification、finite tick trace	nested Resume replay 与 mixed-call composition	单根很强
<code>vTaskSwitchContext</code>	suspended-only refine- ment	unlocked ready selection、任意 ready cursor、public re-entry	部分
<code>vTaskDelay</code>	zero-delay 分支; 固定 artifact-bound P2 pos- itive delay 2	任意 delay/priority/task、blocking list migration、public closure	部分
<code>vTaskDelayUntil</code>	suspended/no-delay 分支	blocking 与 nonblocking all branches、 wrap、public closure	部分

4.1 为什么“2/5”仍需谨慎

- tick-read 的控制流简单, 而且当前 relation 只携带 tick/depth/mask; 返回值证明完整, 但尚不能直接作为 full scheduler public framing 的一根 closure。
- tick 的单根有限 trace 已经是真正通用的重要 milestone, 但 mixed trace 尚不存在, 因而不能把它描述成 “scheduler trace correctness”。
- fixed P2 delay 证明的是一个 artifact-specialised witness, 价值在于 generated-source inhabitation, 而不在于覆盖 arbitrary delay input。

4.2 机器证据摘要

Run	Exit/QAD	总耗时	支撑结论
20260811Tcursor-general-tick-boundary-02	0 / false	249.268 s	public tick boundary closure
20260811Tcursor-general-tick-trace-08	0 / false	261.430 s	arbitrary finite tick-only trace
20260731Tscheduler-tick-refine-03-explicit-model-op	0 / false	24.323 s	xTaskGetTickCount refinement
20260803Tscheduler-delay-zero-03-mod-yield	0 / false	16.508 s	zero-delay modular-yield branch

R · repository 本版 Git 基线为 476e950，与 upstream 为 0 ahead / 0 behind；safe-loop leaf 已由单一 checker lane 以 QAD=false 通过、提交并推送。

5 当前关键接缝：xTaskResumeAll 的 nested-critical replay

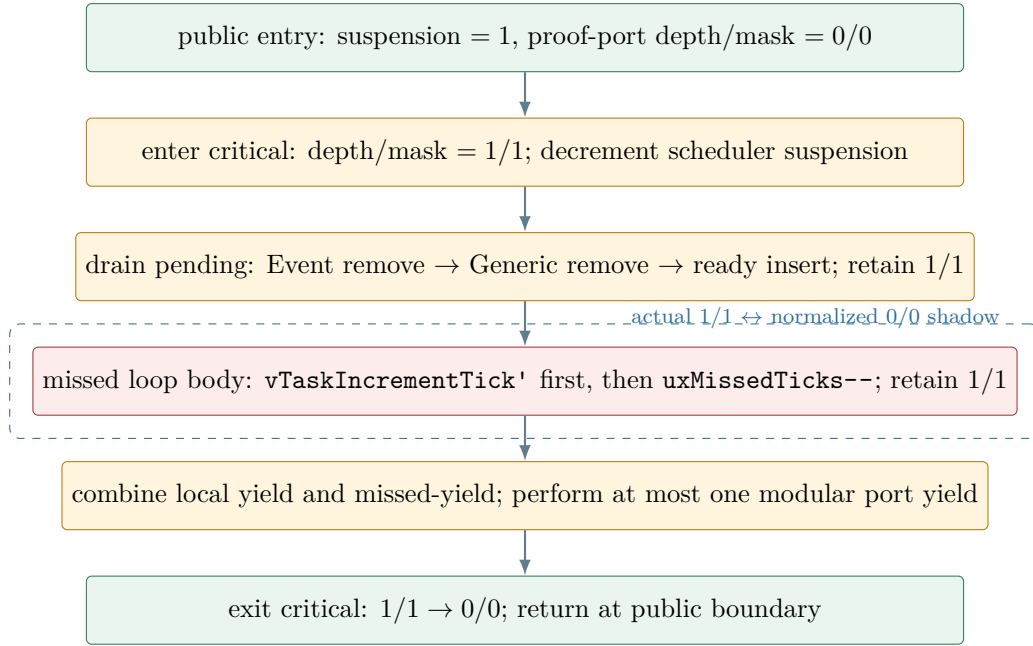


图 2: source-order cutpoint 图。红色步骤不能直接套用 public-boundary tick theorem。

5.1 推荐的语义桥

定义只覆盖 proof-port 两个字段的函数 overlay:

$$\text{overlay}_{d,m}(c_0) = c_0[\text{critical_depth} := d, \text{interrupts_disabled} := m].$$

需要的是 generated tick 对这个函数图关系的 *semantic self-bisimulation*:

$$\text{rel_spec_monad}(c = \text{overlay}_{d,m}(c_0)) (=) \text{tick}' \text{ tick}'.$$

它允许把 normalized shadow 的 checker-green run 迁移到真实 nested state, 并证明输出仍是同一个 overlay。仅有 modifies/frame theorem 不够, 因为“不写这两个字段”并不推出“从不读取或据此分支”。**A · audited**

本版增量: 两字段 overlay algebra、generated unlocked tick 的整条 self-bisimulation、protected 1/1 transport、单次 missed-replay body (先 tick、后 debt decrement)、递归 replay horizon、signed-word arithmetic bridge, 以及 horizon-safe 的完整 generated replay while 均已由 quick_and_dirty=false 的独立 child sessions 检查通过。overlay mismatch 已不再是开放 gate; 当前首个语义缺口是 horizon 不安全时的 first-unsafe no-success 分类。

6 开放 gate 与反例

开放 gate	最小反例/原因	Acceptance criterion
Replay unsafe classification	first-tick=0xFFFFFFFFE、overflow=INT_MAX、debt=2; 第一步成功, 第二步 wrap guard failure	已绿 horizon-safe 成功 while 的补集: 定位首个 unsafe index, 并证明 whole replay loop 无 successful run
Pending frame	port 现有 pending body exact state 保留端口值, 但 loop/outer continuation 弱化后丢失	任意 pending 长度后仍精确保留 running/depth/mask, 并传给 missed-loop continuation
Managed gate	Resume 旧 gate 使用 live-domain 与 live task count; termination retired task 需要 managed-domain	pending task 仍 live, 同时 family、observation、count、termination 和 external roots 全部以 managed 量化
Modular count	yield concrete word MAX_WORD+1=0, 旧 abstract nat SucMAX 不相等	modular-yield scalar relation 经过 final yield 和 public re-entry 保持
Universal delay	zero branch 和固定 P2 delay 不能覆盖任意 delay/task/priority	arbitrary positive delay 的 exact generated state、list migration、yield 与 public closure
Universal switch	suspended-only theorem 不覆盖 unlocked selection 和 ready cursor 推进	all-branch switch theorem 重建同一 common public boundary
Five-root trace	tick-only trace 没有 mixed alphabet	五种 operation 的统一 step relation 与任意 finite enabled sequence induction
Generated concurrency	旧 pure concurrent EnvTick 使用 nat Suc, 且没有 generated-C connection	cursor-general modular environment、linearisation cutpoint、generated source realization 分别 checker-green

6.1 这些为何不是 “proof engineering 小修”

每一项都存在合法输入反例, 或当前 relation 明确丢失源码可观察状态。扩大 timeout、增加 simp、把后态作为前提、固定 cursor、令 managed=live、添加 no-wrap, 均不能消除这些语义反例。它们要求改变 invariant、关系 carrier 或 theorem statement, 然后再进行小步机器检查。 **A · audited**

7 下一 session 的严格执行顺序

序	小 session / rung	Acceptance criterion	首错 fallback
1	C · checked Over-lay/whole tick	graph、run-image、combinator、list callees、while/finally 与整条 source self-bisim 全绿	已完成；不得回退为 modifies-only 论证
2	C · checked Protected tick	实际 depth/mask 原样保留；1/1 nested state 由 normalized public shadow 运输	已完成；不要把内部 cutpoint 改写成 public 0/0
3	C · checked Missed body	正 debt 下严格按源码 tick 后 decrement，得到 exact <code>resume_missed_source_step_abs</code>	已完成；word predecessor 与 snapshot frame 均独立核验
4	C · checked Safe replay while	递归 horizon、signed-word bridge、steps/replay 命名桥与完整成功循环全绿	已完成；主 theorem 仅 entry、quiet、full horizon 三项前提
5	First-unsafe replay	对 horizon 补集给出首个 bad index 与 whole-loop no-success	先补 always-progress/reaches 存在性，再作 first-bad induction
6	Pending port frame	任意 pending drain 后仍为真实 1/1，continuation 获得 frame	用 exact body state + list induction；不重跑大 VCG
7	Managed Resume gate	retired、termination、external roots 与 cursors 保留；pending 任务仍属于 live	先做纯 family/domain bridge，再接 generated body
8	Outer Resume closure	replay、final modular yield、exit critical，all-branch public post 或明确 no-success	分支逐个闭合；禁止把 undefined arithmetic 塞入成功前提

7.1 构建纪律

- 每个 rung 一个独占目录/理论，parent heap 串成 staircase；`parallel_proofs=0`。
- 只开一个 Isabelle build lane；`quick_and_dirty=false`。
- timeout 时先查 session DB / command timing，插入 deliberate checkpoint 二分；只有已测得冷缓存有界后才提高 timeout。
- 每次只修第一条 checker 错误；禁止 broad simp 展开大型 family/snapshot conjunction。
- 新定理先做 adversarial counterexample audit，再进入 ROOT。

8 可以公开写什么，以及绝不能写什么

8.1 可公开陈述

“在冻结的 FreeRTOS V6.1.1 artifact-specialised CParser/AutoCorres2 语义中, Isabelle/HOL 已经以 `quick_and_dirty=false` 核验了 `cursor-general` 的 `generated vTaskIncrementTick` 公共边界闭包, 并证明任意有限次、逐步 `arithmetic-enabled` 的重复 `tick` 调用精确对应 `modular abstract iteration`。”

“任意有限 `due prefix` 的 `generated while/finally` 已覆盖 `empty`、`future-exception`、`singleton` 及 `arbitrary due-list` 控制流; `managed task domain` 可严格大于 `runnable live domain`。”

8.2 绝不能公开陈述

- “FreeRTOS V6.1.1 scheduler 已被完整形式化验证。”
- “五个根函数对所有合法输入均已闭合。”
- “任意 scheduler API trace 已证明。”
- “并发中断 `linearisation` 已由 `sequential runs-to theorem` 推出。”
- “从 `boot/allocator` 构造的所有可达状态都满足当前 `public boundary`。”
- “45% 是 Isabelle 自动计算或仓库原生的完成率。”
- “`public boundary` 完全 `ghost-free`。”更准确的说法是: 调用方不显式传 `witness`; 证明内部仍可存在 `existential proof witnesses`。

8.3 建议脚注模板

“本文的完成率按三个明确口径给出: 严格 `acceptance closure` 约 35–40%, 可复用证明基础设施约 45–50%, 以及尚待逐项命名 A1–A8 的粗粒度台账 $9/20 = 45\%$ 。三者均不按 `theory`、`lemma` 或 `session` 数量计数, 也不属于 Isabelle 定理。checker-green 仅表示列出的 `session` 在 Isabelle2025-2 下以 `quick_and_dirty=false` 完成; 未注册草稿、外部 Pro 建议、测试与手工 `trace` 均不计入已证覆盖。”

8.4 可信工具链边界

- 冻结 ELF、layout ledger、generated address configuration 与其 hash 是外部构建证据; ELF-to-config correspondence 没有被内部化为 Isabelle theorem。
- CParser、AutoCorres2、Isabelle kernel、编译器/链接器属于显式 `trusted/toolchain boundary`。
- 外部 Pro 只做叙述、反例和路线审阅; 其答案不能替代 `kernel-checked theorem`。
- 接受的 `proof sessions` 不得包含 `sorry`、`oops`、`admitted axiom`、`oracle shortcut` 或 `quick_and_dirty=true`。

9 证据附录与交接摘要

9.1 版本与 run 证据

项目	值
Baseline HEAD	476e950263a75f6d64542453c2b3bb6b31fecb7a
Upstream sync	ahead 0 / behind 0 (safe-loop 提交后)
Boundary run	20260811Tcursor-general-tick-boundary-02, exit 0, QAD false
Trace run	20260811Tcursor-general-tick-trace-08, exit 0, QAD false
Nested overlay capstone	20260811Tnested-tick-overlay-capstone-01-cold, exit 0, QAD false
One replay body	20260811Tnested-tick-missed-body-capstone-02-explicit-tail, exit 0, QAD false
Replay horizon/bridge	20260811Tnested-tick-replay-horizon-abs-03-signed-word / 20260811Tnested-tick-replay-arithmetic-bridge-11-signed-word, exit 0, QAD false
Safe replay while	20260811Tnested-tick-replay-safe-loop-01-cold, exit 0, QAD false, 147.663 s
Tracked scale (非完成率)	393 tracked .thy; 333 ROOT sessions; 63 cursor-general sessions
Public PR	https://github.com/hellotommy/freertos-v611-scheduler-isabelle/pull/2

9.2 下一 session 必读摘要

目前最强增量已经越过 nested/public mismatch: generated tick 对 proof-port overlay 的无前提 self-bisimulation 已绿，因此真实 1/1 replay cutpoint 可以复用 normalized 0/0 shadow; 单次 body、全债务 horizon、signed-word arithmetic bridge 与 horizon-safe 完整 replay while 也已绿。下一 session 不得从旧 probe 重来，而应先证明 horizon 补集的 first-unsafe no-success: 由于 runs_to 本身不保证存在可达 successor，须先给 replay body 建立 always_progress/reaches 支点，再按首个 bad index 归纳。随后强化 pending drain 保留真实 1/1 与 running frame，另建 cursor-general managed Resume gate (pending task 仍属于 live，而 decoder/family/count 覆盖 managed 与 termination)，最后以 modular yield relation 闭合 outer exit。不能以 no-wrap、cursor=None 或 managed=live 排除合法状态。当前仍没有 universal Resume、mixed five-root trace 或 generated concurrency theorem。

9.3 报告生成与审阅说明

本报告由本地仓库、绿色 run status、当前 theorem statements 与 source-order audit 汇总; 外部 Pro 复核了完成率分母、claim language、开放反例和八级 continuation staircase。该路线的 overlay、protected tick、单次 body 与 safe replay 成功分支现已由本地 checker 落地; Pro 建议的 20-gate 台账仍为 $9/20 = 45\%$ ，因为尚未跨过新的顶层五根 acceptance gate。其同时指出 A1–A8 尚未逐项命名，且 xTaskGetTickCount 当前 relation 尚无 full scheduler framing。因此本报告把 45% 限定为暂定工程台账，并把严格 common-boundary closure 仍报为 35–40%。最终文本只采纳与仓库及 checker evidence 相容的建议。 [R · repository](#)